OPERATING SYSTEMS – ASSIGNMENT 1

TASK 5 – Examine System Calls (4 Marks)
CO1–CO5 | Bloom's Level: K5 – Evaluate

Introduction

A system call is the method by which a user program requests a service from the operating-system kernel. Applications cannot directly perform privileged hardware operations, so they use system calls for controlled access.

1. Categories of system calls

Process control: create, execute and terminate processes; examples include fork(), exec() and wait().
File management: create, open, read, write and close files; examples include open(), read(), write() and close().
Device management: request or release devices and perform device I/O.
Information maintenance: obtain or update system information such as process IDs, time and attributes.
Communication: exchange data between processes or computers using pipes, sockets and shared memory.
Protection: control permissions and access rights, for example through permission-related system calls.

2. What happens when a program calls read()

1. The application calls the read() library wrapper with a file descriptor, buffer address and number of bytes.
2. The wrapper places the system-call number and parameters in the required locations.
3. A special trap or system-call instruction transfers control from user mode to kernel mode.
4. The kernel checks the parameters and verifies that the process is allowed to read from the file descriptor.
5. The kernel performs the required file or device operation and transfers available data to the user buffer.
6. The kernel places the result or error information in the return location and switches back to user mode.
7. The program continues after read() and checks the returned number of bytes.

3. How parameters are passed to the kernel

System-call parameters may be passed through CPU registers, through a block or structure stored in memory whose address is placed in a register, or through the stack. The exact method depends on the processor architecture and operating-system calling convention.

4. Using strace

The Linux command **strace** displays system calls made by a program. For example, compile a C program and run **strace ./program**. The output can show calls such as openat(), read(), write() and close(), along with their arguments and return values. The command **strace -c ./program** provides a summary of system-call counts and time. This helps identify how an application uses OS services.

5. Alternative concept: C library functions vs direct system calls

A program may use fopen(), fread() and fwrite() from the C standard library instead of directly calling open(), read() and write(). The library can maintain a user-space buffer. It may collect many small application reads into a larger read() system call, reducing transitions into the kernel.
For a file of n bytes and a library buffer of b bytes, the approximate number of read operations can be around n/b, plus calls for end-of-file and other handling. With a 4096-byte buffer, the number of data reads is approximately n/4096. Without buffering, small reads may cause many more system calls. Library buffering requires extra memory, usually around the buffer size per stream, but can improve performance.

6. Evaluation and conclusion

Direct system calls give the programmer more control but can cause frequent kernel transitions if used for small amounts of data. Buffered library functions are usually more convenient and can reduce system-call overhead for ordinary file processing. For hospital applications, the correct choice depends on performance, reliability and the need for precise I/O control.

Conclusion

System calls provide a safe interface between applications and the kernel. Understanding their categories and execution steps helps explain how hospital programs access files, devices and communication services.

Prepared from the corresponding task in the uploaded CSA04 Operating Systems Assignment 1 document. Add your name, register number, required screenshots and your own measured evidence wherever requested before submission.